BUILD • BREAK • ENGINEER

Hands-On Technical Mentorship Program

Learn differently. Learn the hard way. See the difference.

TRACKS

Track 1 – Programming: Under the Hood (C) · Built to Last (Rust)

Track 2 – Security: Think Like an Attacker · Break the App · Inside the Beast · Own the OS · Secure the Cloud · Shift Left · Red Team, AI Edge

Track 3 – AI: AI That Ships

Opening Summer Mentorship Slots – Only for College Graduates

WHAT THIS PROGRAM IS

This is not online training. No one will teach you from slides or run you through labs. You learn by doing, getting stuck, and then working through it. That is the difference. Guidance is tailored to each individual. With more than two decades in the industry, the mentor knows what the industry expects; these programmes are aimed at that.

Programming: You will not learn by watching. You will write real code and build real systems in C and Rust.

Security: You will not only study vulnerabilities. You will find and fix them in networks, apps, malware, cloud, and pipelines. Think like an attacker; defend like an engineer.

AI: You will not just watch demos. You will build with real models and ship AI that works.

PROGRAM DESIGN

You attempt the problems first. You struggle with the code, tools, or techniques. Then we discuss the approach, correct mistakes, and improve the solution. The structure is there; the hard work is yours. We recommend 120 to 150 days for good coverage of a programme and completion of the capstone project.

The difference comes from:

- Real-world problem statements you tackle yourself
- Direct technical discussions after you have tried
- Debugging and troubleshooting your own code
- Review of your work and progress when you have something to show

This is not an online program. This is how you learn differently.

CERTIFICATION

Certificates of Completion are issued only after the capstone project is completed and demonstrated. This program is not attendance-based.

Certification requires:

- Completion of the assigned capstone project
- Technical demonstration of the work

- Clear understanding of the technical implementation

No capstone. No certification.

This is just the gist. Enroll to gain the full depth.

Programmes

Ten programmes across three tracks. Hands-on.

1. **Under the Hood**

   C Programming & System Programming

   Write real code. Debug real crashes. Own the machine. Build utilities that run at the core of systems. Build a mini shell, log parser, or packet sniffer.

2. **Built to Last**

   Rust & Modern Systems Development

   Safe, fast, reliable. No garbage collector, no compromise. Backend services and CLI tools that ship. Build a production-style API or async scanner.

3. **Think Like an Attacker**

   Network & Server Penetration Testing

   Find what's exposed. Pivot. Escalate. Document. Real infrastructure, real attack chains, lab-only. Deliver a full engagement report or enum toolkit.

4. **Break the App**

   Application & Web Security Testing

   Auth, APIs, logic flaws. Break them before attackers do. Manual testing and professional reports. Build an app, then break it and fix it.

5. **Inside the Beast**

   Malware Analysis & Reverse Engineering

   Static and dynamic analysis. PE, assembly, IOCs. See how malware really behaves in a safe lab. Reverse a sample and document the kill chain.

6. **Own the OS**

   Secure Systems - Linux, BSD & Windows

   Harden and reason about Linux, BSD, and Windows. Attack surface, configuration, scripting on real systems. Deliver a hardening guide or secure baseline.

7. **Secure the Cloud**

   Cloud Security

   IAM, misconfigs, network. Build secure, find weak. AWS, Azure, or GCP in lab environments. Design secure architecture or deliver an assessment report.

8. **Shift Left**

   DevSecOps / Secure SDLC

   SAST, SCA, pipelines. Security in the build. Fix findings, reduce risk before production. Secure a pipeline and document the journey.

9. **Red Team, AI Edge**

   Classical Red Teaming Using AI

   Phish, macros, C2. AI speeds the build; you run the op. Authorised only. Lab environments, clear scope. Deliver a full red team report with AI-assisted phases.

10. **AI That Ships**

    Hands-On AI & Systems Engineering

    RAG, agents, local LLMs. Real models, real products. From prototype to something users can run. Build a RAG app, multi-agent workflow, or AI pipeline.

Under the Hood

C Programming & System Programming

Build real system utilities. Debug crashes and memory problems. Understand how programs actually run.

THEME

Understanding how computers actually execute programs. Focus on memory behavior, pointers, debugging, system interaction, and writing efficient system-level code.

CORE TOPICS

- C program structure and compilation model
- Memory layout (stack vs heap)
- Pointers and pointer arithmetic
- Dynamic memory management (malloc / free)
- File I/O and system interaction
- Modular programming and multi-file projects
- Debugging techniques and program tracing
- Implementing core data structures directly in C
- Socket programming fundamentals (TCP / UDP basics)

REAL SYSTEM IMPLEMENTATION

You will implement core data structures and algorithms directly in C. These components will be integrated into small system utilities that process files, network data, or system input. Typical concepts explored include: linked list structures used in low-level system components; hash tables for fast lookup; buffer management techniques; file indexing approaches. You will implement these components yourself, integrate them into working programs, and debug them when they fail.

SAMPLE CAPSTONE PROJECTS

- Terminal-based text editor (similar to a simplified nano)
- Secure file transfer utility for Linux systems
- Mini shell for Linux command execution
- High-speed log parser for large log files
- Packet sniffer using libpcap (tcpdump-style)
- Simple network port scanner

- Configuration file parser and validator

Built to Last

Rust & Modern Systems Development

Build high-performance systems. Write memory-safe concurrent programs. Design reliable backend and system tools.

THEME

Building high-performance systems with memory safety. Focus on ownership, concurrency, and reliable systems programming.

CORE TOPICS

- Rust program structure and Cargo ecosystem
- Ownership and borrowing model
- Lifetimes and memory safety
- Error handling patterns (Result / Option)
- Modular project structure and crate design
- Concurrency and asynchronous programming (Tokio)
- Building command-line tools and system utilities
- Backend services and REST APIs in Rust

REAL SYSTEM IMPLEMENTATION

You will build real tools and services using Rust. Programs will focus on reliability, performance, and safe concurrency. Typical concepts explored include: safe memory management using the ownership model; concurrent task execution using async runtimes; efficient request handling in backend services; data processing pipelines with controlled resource usage.

SAMPLE CAPSTONE PROJECTS

- Production-style backend service with REST APIs and database integration (Axum / Actix)
- High-performance asynchronous network scanner using Tokio
- Terminal-based system monitoring dashboard using Ratatui
- Concurrent file processing pipeline using async Rust
- Network traffic analysis tool using asynchronous packet processing
- AI inference API service for high-performance request handling

Think Like an Attacker

Network & Server Penetration Testing

Discover exposed services and hidden attack paths. Analyze systems the way attackers do. Understand how real infrastructure gets compromised.

THEME

Understanding how attackers discover weaknesses in networks and server infrastructure. Focus on reconnaissance, service analysis, exploitation techniques, and documenting security weaknesses.

CORE TOPICS

- Network discovery and host enumeration
- Service identification and attack surface analysis
- Linux and Windows server attack surfaces
- Credential attacks and lateral movement concepts
- Privilege escalation techniques
- PowerShell and shell environments
- Manual penetration testing workflow
- Vulnerability documentation and reporting

REAL ATTACK ANALYSIS

You will analyze how weaknesses in networks and servers are discovered and exploited. Exercises focus on identifying exposed services, misconfigurations, and privilege escalation paths. Typical areas explored include: service enumeration and version analysis; authentication weaknesses; misconfigured services and permissions; privilege escalation paths. Students reproduce attack paths in controlled lab environments and document the full attack chain.

SAMPLE CAPSTONE PROJECTS

- Simulated enterprise network compromise with full attack chain documentation
- Multi-host penetration testing scenario
- Server privilege escalation research and exploitation report
- Automated network enumeration toolkit
- Network attack simulation with complete security assessment report

Break the App

Application & Web Security Testing

Break authentication and access controls. Analyze how modern applications fail under attack. Understand the real impact of insecure design.

THEME

Understanding how web applications fail under attack. Focus on authentication, authorization, APIs, logic vulnerabilities, and analyzing the real-world impact of insecure application design.

CORE TOPICS

- Web architecture and HTTP fundamentals
- Authentication mechanisms and session management
- Authorization and access control failures
- REST API security testing and abuse scenarios
- Injection vulnerabilities and input handling failures
- Business logic vulnerabilities
- Manual web application testing workflow
- Vulnerability documentation and secure design practices

REAL APPLICATION ANALYSIS

You will analyze how modern web applications expose vulnerabilities. Exercises focus on identifying weaknesses in authentication flows, API endpoints, and application logic. Typical areas explored include: authentication bypass scenarios; access control weaknesses; API endpoint misuse; logic flaws in application workflows.

SAMPLE CAPSTONE PROJECTS

- Build a full-stack web application and analyze its security weaknesses
- Authentication and authorization bypass research project
- REST API abuse and security testing scenario
- Business logic vulnerability analysis
- Full vulnerability assessment of a web application with a professional security report

Inside the Beast

Malware Analysis & Reverse Engineering

Analyze how malicious software behaves. Understand how malware evades detection. Learn how attackers design malicious code.

THEME

Understanding how malicious software works internally. Focus on reverse engineering, behavioral analysis, and identifying how malware interacts with operating systems.

CORE TOPICS

- Malware execution flow and attack stages
- Static malware analysis techniques
- Dynamic malware analysis techniques
- Portable Executable (PE) file structure
- Assembly fundamentals for malware analysts
- Sandbox environments and behavioral monitoring
- Indicators of compromise and artifact analysis
- Malware documentation and reporting

REAL MALWARE ANALYSIS

You will analyze how malware operates within controlled analysis environments. Exercises focus on identifying execution behavior, persistence mechanisms, and extracting indicators of compromise (IOCs). Typical concepts explored include: process execution and memory behavior; file system and registry modifications; network communication patterns; extraction of IOCs; process injection and process hollowing. Students analyze these behaviors in controlled lab environments and document the technical findings.

SAMPLE CAPSTONE PROJECTS

- Reverse engineer a malware sample and document execution behavior
- Ransomware behavior analysis report
- Malware analysis lab and sandbox environment setup
- Malicious document attack chain analysis

Own the OS

Secure Systems - Linux, BSD & Windows

Harden and understand Linux, BSD, and Windows from a security and systems perspective. Attack surface, configuration, scripting, and how things actually work on each OS. Build and break real systems across the three platforms.

THEME

Understanding how to secure and reason about Linux, BSD, and Windows at the systems level. Focus on hardening, secure configuration, attack surface, and misconfigurations. You will work with real installations (lab or VM) to harden systems, find weak spots, and document what matters.

CORE TOPICS

- Security fundamentals across Linux, BSD, and Windows
- Hardening and secure configuration (services, permissions, auth)
- Attack surface and common misconfigurations
- User and privilege model; authentication and access control
- Scripting and automation for security and ops (shell, basic automation)
- Logging and audit (what to enable, where to look)
- Comparing security posture and trade-offs across the three OSes
- Network and service exposure; firewall and access policies

REAL SYSTEMS WORK

You will work with real Linux, BSD, and/or Windows systems in lab or VM environments. Exercises focus on hardening systems, identifying misconfigurations, and documenting findings. Typical areas: unnecessary services and open ports; weak or default credentials; permission and privilege issues; logging and audit configuration.

SAMPLE CAPSTONE PROJECTS

- Hardening guide or hardened image for one or more of Linux, BSD, Windows
- Attack surface assessment and remediation report for a lab system
- Cross-platform automation or hardening script set (e.g. baseline checks)
- Secure baseline documentation for a specific use case (e.g. server, workstation)
- Comparison report: same workload secured on Linux vs BSD vs Windows with recommendations

Secure the Cloud

Cloud Security

Secure and assess cloud environments (AWS, Azure, GCP). Understand identity, misconfigurations, and attack paths. Build and break real cloud setups.

THEME

Understanding how cloud infrastructure is secured and how it is attacked. Focus on identity and access, misconfigurations, network exposure, and secure design. You will work with real cloud providers in lab environments to build secure patterns and to find and fix weaknesses.

CORE TOPICS

- Cloud provider fundamentals (identity, regions, networking)
- Identity and access management (IAM, roles, policies, federation)
- Common misconfigurations and insecure defaults
- Storage and database security in the cloud
- Network security (VPCs, security groups, segmentation)
- Container and serverless security basics
- Cloud security assessment and reconnaissance
- Secure architecture patterns and hardening

REAL CLOUD SECURITY WORK

You will work with real cloud environments in lab or personal accounts. Exercises focus on building secure configurations, identifying misconfigurations, and documenting findings. Typical areas: overly permissive IAM; exposed storage or APIs; network misconfigurations; insecure identity patterns.

SAMPLE CAPSTONE PROJECTS

- Secure multi-account cloud architecture design and implementation
- Cloud security assessment report (IAM, storage, network) for a lab environment
- Automated misconfiguration scanner or checker for a cloud provider
- Incident response runbook for a cloud-based workload
- Comparison of secure patterns across AWS / Azure / GCP with recommendations

CLOUD & INFRASTRUCTURE NOTE

This track requires access to at least one major cloud provider (AWS, Azure, or GCP). You will need a cloud account for exercises and capstones. Free tier or trial accounts are often

sufficient; some usage may incur cost beyond free tier. Participants should be prepared to manage minimal cloud usage for their own practice and projects.

Shift Left

DevSecOps / Secure SDLC

Build security into the development pipeline. Use SAST, DAST, and supply chain practices. Ship code that is secure by design.

THEME

Understanding how security is integrated into the software development lifecycle. Focus on secure SDLC, CI/CD security, automated testing (SAST, DAST, SCA), supply chain security, and secure coding practices. You will work with real pipelines and tools to add security checks and fix findings.

CORE TOPICS

- Secure SDLC and shift-left concepts
- CI/CD fundamentals and pipeline security
- Static application security testing (SAST) and code analysis
- Dynamic testing and dependency/SCA (software composition analysis)
- Supply chain security (dependencies, containers, signing)
- Secure coding practices and remediation
- Security gates and policy as code
- Vulnerability management and prioritisation in development

REAL PIPELINE AND CODE SECURITY

You will work with real or sample codebases and pipelines. Exercises focus on adding security tooling, fixing reported issues, and understanding trade-offs. Typical areas: integrating SAST or SCA; fixing findings and reducing false positives; securing build and deploy; reviewing dependencies and upgrade policies.

SAMPLE CAPSTONE PROJECTS

- Add security tooling (SAST/SCA) to an existing CI/CD pipeline and document findings
- Secure a sample application and its pipeline end to end with a written report
- Supply chain security assessment (dependencies, container image) with recommendations
- Custom security gate or policy (e.g. branch protection, image signing) with documentation
- Vulnerability management playbook for a development team

Red Team, AI Edge

Classical Red Teaming Using AI

Run real red team engagements. Use AI to build phishing lures, generate macros, and create C2 and implants. Learn classical attack chains with AI accelerating the build.

PREREQUISITES

This track requires access to commercial AI models. Participants should subscribe to commercial LLM services as needed to create artefacts, design and implement C2s, and complete the hands-on exercises.

THEME

Classical red teaming (phishing, initial access, C2, implants, persistence, lateral movement) using AI to generate and refine the technical artefacts. Focus on real engagement tradecraft: phishing and macro-enabled payloads, command-and-control design, implant and agent development. AI is used as a force multiplier for generating lures, VBA/macros, C2 components, and supporting code. All work is conducted in controlled lab environments with clear scope and authorisation.

CORE TOPICS

- Red team methodology: objectives, scope, rules of engagement, reporting
- Phishing and social engineering: lures, pretexting, AI-generated copy and scenarios
- Macro generation: VBA and Office macros for initial access, AI-assisted generation and obfuscation
- Command-and-control (C2): design, protocols, implants, and AI-assisted C2 creation
- Payload and implant development: shellcode, loaders, and LLM-assisted scaffolding
- Evasion and detection: bypassing AV/EDR, trade-offs when using generated code
- Post-exploitation and persistence in classical red team contexts
- Responsible use, authorisation, and controlled environments

REAL RED TEAM OPERATIONS

You will run classical red team exercises and use AI to build the artefacts. Exercises focus on full attack chains: phishing and initial access (including macro-based), C2 setup, implant deployment, persistence, and exfiltration or impact. You will use AI to generate phishing content, macros, C2 agents or stagers, and supporting code - then test, refine, and document in lab environments. Typical areas explored include: phishing campaigns with AI-generated lures and macro-enabled documents; VBA and Office macro generation (and obfuscation) with AI assistance; building or adapting C2 frameworks and implants with AI-assisted code generation; payload and loader development with LLM-assisted generation and safe review; end-to-end red team scenario from phishing through to report.

SAMPLE CAPSTONE PROJECTS

- Full red team engagement: phishing (with AI-generated lures and macros), C2, and implant deployment with written report
- AI-assisted phishing and macro generation toolkit or playbook for authorised testing
- C2 and implant creation using AI-assisted code generation (lab environment)
- Macro-enabled initial access chain (document + macro + callback) with AI-generated components
- Red team tooling suite: phishing, macro generation, and C2 creation with documentation

ETHICS AND SCOPE NOTE

This track is for authorised red teaming only. All exercises and capstones are conducted in controlled environments, with explicit scope and rules of engagement. AI is used to build classical red team artefacts (phishing, macros, C2, implants) within legal and ethical boundaries.

AI That Ships

Hands-On AI & Systems Engineering

Build practical AI-powered tools. Work with real models and real data. Design AI systems that solve real problems.

THEME

Understanding how modern AI systems are designed and engineered for real-world applications. Focus on building AI-powered tools, working with local models, creating agentic workflows, and designing practical AI systems that interact with real data and applications.

CORE TOPICS

- Working with open-source large language models
- Running local models using tools such as Ollama
- HuggingFace ecosystem and model usage
- Retrieval-Augmented Generation (RAG) systems
- Agentic workflows using frameworks such as LangChain and CrewAI
- Designing AI pipelines that process documents and structured data
- Responsible AI concepts and system-level risks

SAMPLE CAPSTONE PROJECTS

- Build a RAG application using local LLMs and vector databases
- Develop a multi-agent workflow coordinating multiple AI agents
- Build a local LLM application using Ollama and open-source models
- Create an AI-powered knowledge assistant using HuggingFace models
- Design an AI pipeline that processes documents and answers contextual queries

HARDWARE & INFRASTRUCTURE NOTE

This track may require stronger hardware. Depending on the project, students may run models locally or use cloud-based AI platforms. Typical environments: local model execution (e.g. Ollama); systems with sufficient RAM or GPU; cloud platforms such as AWS Bedrock, AWS SageMaker, or similar.

Join the Program

Mentorship, Trainer & Enquiry

LIMITED MENTORSHIP PROGRAM

This program is intentionally limited in size. Paid program; monthly participation fee. No stipend. You work on real technical problems; when you are stuck or have something to show, we engage. No one teaches you step by step. You learn the hard way; we discuss, review, and course-correct. That is why the program is limited to a small number of participants.

You get:

- Discussion after you have attempted the problem
- Review of your code, projects, and implementation
- Structured problem tracks to work through
- Feedback on your approach, debugging, and decisions

INDUSTRY EXPOSURE & GUIDANCE

Beyond the programmes, the mentor shares perspective from over two decades in cybersecurity and software: how real engineering and security teams operate, and what matters when you are building or breaking systems.

Participants may receive:

- Recommendations of important technical books and research material
- Guidance on useful conferences, communities, and professional learning resources
- Advice on building strong technical portfolios and project documentation
- Insight into industry expectations during technical interviews

Students who demonstrate strong effort, curiosity, and technical discipline may also receive guidance on preparing for industry opportunities and career direction.

ABOUT THE TRAINER

Founder – Tensor42 Technologies

Involved in Cybersecurity and Programming since 1994. Working in the Cybersecurity industry since 2003.

Professional experience includes working with a major Fortune 10 organization and a leading global antivirus company. Hands-on exposure across offensive security, malware analysis, application security, and secure systems development. Has worked with 50+ students and professionals who learned by doing and built strong technical foundations.

ABOUT TENSOR42

Tensor42 Technologies builds products in cybersecurity, AI, and software. Active development includes security tooling, red team and defensive platforms, and AI-assisted workflows. The mentorship runs alongside real product work, so participants get exposure to how commercial tools and systems are built.

APPLICATION & ENQUIRY

LinkedIn
https://www.linkedin.com/in/senthilvelan/

Email
senthilvelantraining@gmail.com

This is just the gist. Enroll to gain more.

This program is designed for students who want real technical depth and practical engineering skills.

Disclaimer: Some of the wording here was drafted with AI assistance. The topics, flow, themes, how deep the piece goes, overall structure, and editorial style are mine: I

decided what to cover, in what order, and how to say it.

kkvelan · built with GitHub Pages